TITLE AND DATE

Document name: Project#4 Motor Driven Fan Lab Report
Document reference: kiddKid.cmpe311.fall26.project#4
Date of publication: October 30, 2026

LEAD ENGINEER: William Wang

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA
MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project document for Project#1 of the CMPE311 class. It contains customer, technical and testing requirements as well as the design and results of the validation testing.

DESCRIPTION: This design is to create an embedded system based upon an Arduino Uno R3 development platform that allows the user of the program to specify an LED number and its blink interval. The blinking of the LEDs must continue asynchronously without any additional input from the user. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation.

RESULT SUMMARY: The project was a success, the embedded system design meets all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES:

- ProjectAsynTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Spring 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

***DEFINITIONS*:**

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The Requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Educational-constraints” – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User.

“The Reference-platform” – The configuration of The System used by The Developer to test and validate

“PROJECT-ASYNC”, “PROJECT-CE” – The previous projects upon which this project is based

“arduino.h” - helper functions for the Arduino development platform maintained by Arduino

The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

AG, owned by Qualcomm.

IP – Intellectual Property

ISR – Interrupt Service Routine

LED – Light Emitting Diode

The Product/System – The device being designed and prototyped

PROJECT-MYMILLI – The previous project upon which this project is based

PWM – Pulse Width Modulation

RPM – Revolutions Per Minute

REQUIREMENTS (Part I)

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement
May – your design may satisfy the requirement but doesn't have to
Informative – the intent of the following description is to make the requirement more understandable
All customer requirements are started with "C.#".
All high-level technical requirements are started with "HL.#".
All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS:

The customer requirements, C1 through C6, are identical to that of PROJECT-MYMILLI. See

CUSTOMER REQUIREMENTS section.

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.5, are identical to that of PROJECT-MYMILLI. See HIGH-LEVEL TECHNICAL REQUIREMENTS section.

REQUIREMENTS (Part II)

Customer Requirements:

C6. The device must have a motor-driven fan
C7. The RPM of the fan must be controllable via a push buNon
C8. The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH
C9. Each hit of the buNon must cycle the speeds from OFF to LOW, MEDIUM, HIGH, MEDIUM, LOW, OFF
C10. LOW must be able to move air with minimal sound and minimum feeling of flow
C11. MEDIUM must be able to move air with moderate sound and a moderate feeling of air flow
C12. HIGH must be able to move air quickly with moderate sound and a distinct feeling of air flow

HIGH-LEVEL TECHNICAL REQUIREMENTS:

The high-level technical requirements, HL.1 through HL.6, are identical to that of PROJECT-MYMILLI. See HIGH-LEVEL TECHNICAL REQUIREMENTS section.
HL.7 The device must have a DC motor-driven fan with an opera]ng voltage between 3V and 12V
HL.8 The fan must operate at a minimum between 1500 and 6500 RPM
HL.9 The fan must draw less than 4 Amps
HL.10 The fan must be driven by a PWM signal with a maximum voltage of 5VDC
HL.11 The fan must have a maximum dimension less than 4”x 4”x 3” (not including cage)
HL.12 The RPM of the motor must be controllable using a single-pole, momentary-on, normally-off push buNon.
HL.13 The opera]on of the buNon and motor must have no noticeable effect upon the opera]on of previously implemented I/O (e.g. serial monitor and LEDs)
HL.14 The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH
HL.15 Each hit of the buNon must cycle the speeds from OFF to LOW to MEDIUM to HIGH to MEDIUM to LOW to OFF
HL.16 LOW must be able to move air with a sound volume less than 35dB and airflow levels between 30-45 (deleted 4/1/2030 twk) 0.30 to 0.45 CFM ± 0.05 CFM
HL.17 MEDIUM must be able to move air with a sound volume less than 40dB and airflow levels between 45-70 (deleted 4/1/2030 twk) 0.45 to 0.70 ± 0.05 CFM
HL.18 HIGH must be able to move air with a sound volume less than 50dB and airflow levels between 70-100 (deleted 4/1/2030 twk) 0.70 to 1.00 ± 0.05 CFM

EDUCATIONAL RESTRAINTS REQUIREMENTS:

E.4 The Evaluator must be able to successfully have The System perform the dialog in Table 4 and Table 5. (added – kidd, 3/20/2030)

E.5 H.16, H.17, H.18 do not have to be explicitly measured since we don't have the necessary testing equipment. Just provide some evidence that these requirements have been satisfied though justified inference such as through calculations.

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITE 2 above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

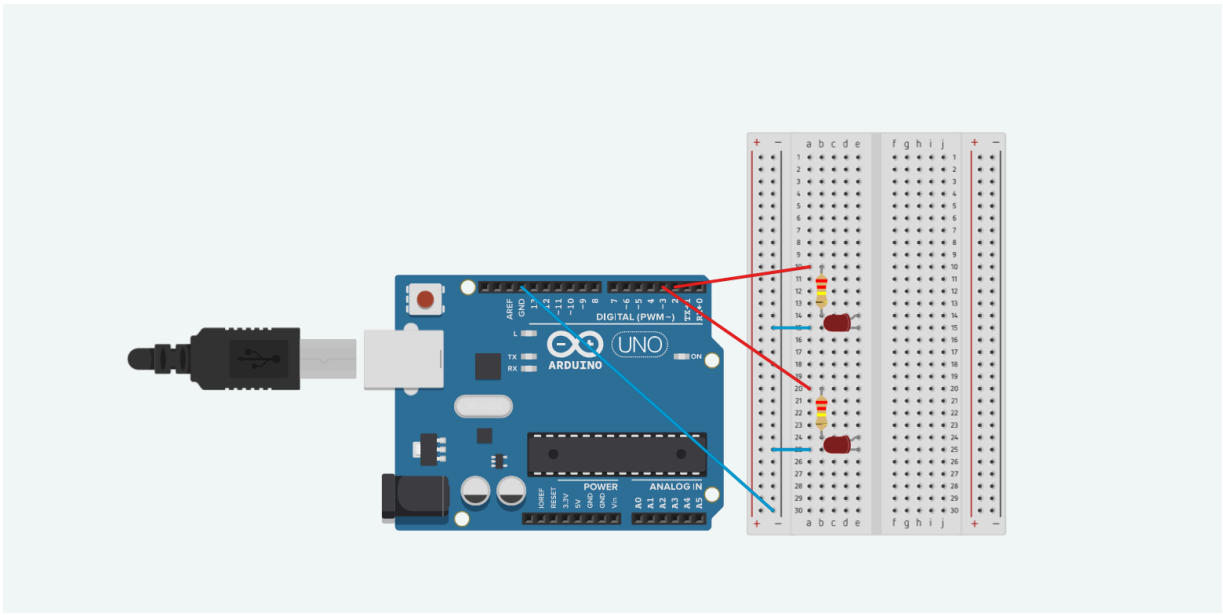
1. The program uses a round-robin cyclic executive task manager with a function pointer array

See Appendix A for the logical flow of the project program.
See Appendix B for the code executing on the Arduino Uno R3.

TESTING AND VALIDATION

DESCRIPTION: The tests that have to be performed and validated are shown in Table 2. These tests were performed on the testbed shown in Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system though testing must be more thorough than this. The results of testing are shown in Table 3. When necessary, a video of the test is provided along with this report.

Figure 1. Testbed Setup. LEDs connected to digital pins 2 & 3.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly to Digital Outputs

Table 1. Required Test Dialog (***blue*** are the user inputs).

Serial port I/O	**Notes**
What LED? (1 or 2) ***2***	
What interval (in msec)? ***600***	LED2 starts blinking at an interval of 300ms on and 30,0ms off. LED1 is unaffected.

What LED? (1 or 2) ***1***	
What interval (in msec)? ***1600***	LED1 starts blinking at an interval of 8800ms on and 800ms off. LED2 is unaffected.

Table 2. Required Test Dialog (***blue*** are the user inputs).

Button	**Fan**
Initial state: button unpressed	Initial State: Fan off
Press	Fan low
Press	Fan medium
Press	Fan high
Press	Fan medium
Press	Fan low
Press	Fan off
Repeat sequence above	

TESTING AND VALIDATION REQUIREMENTS:

Table 3. Testing and Validation Requirements

The testing and validation requirements, T.0 through T.9, are identical to that of PROJECT-MYMILLI. See TESTING AND VALIDATION REQUIREMENTS section.

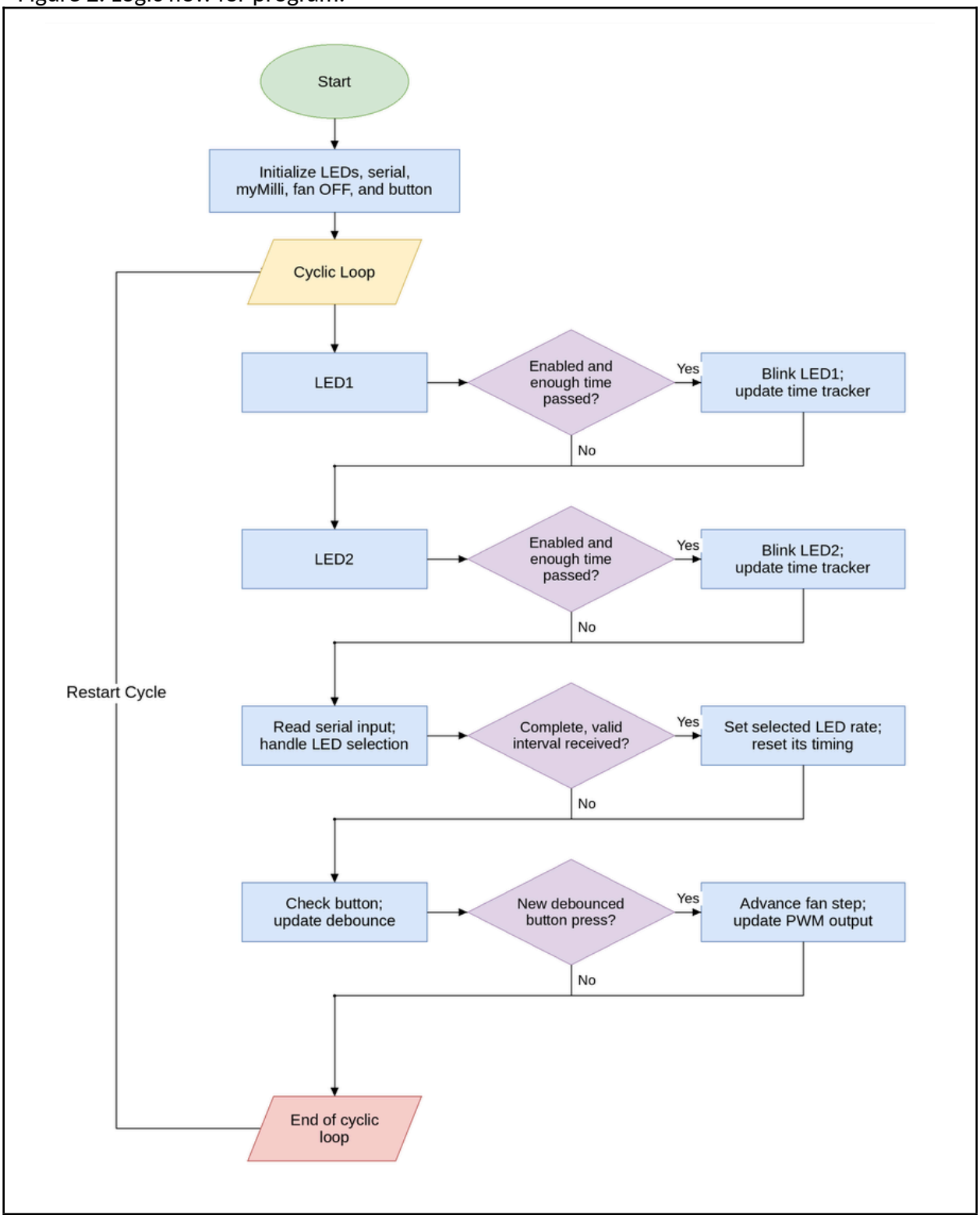
TESTING RESULTS:

Table 4. Results of tests

Test performed	Results
T.0	Satisfied. See video of test.
T.1	Satisfied. See video of test.
T.1.A	Satisfied. See video of test.
T.2	Satisfied. See video of test.
T.3	Satisfied. See video of test.
T.4	Satisfied. See video of test.
T.4.1	Satisfied. See video of test.
T.5	Satisfied. See video of test.
T.6	Satisfied. See video of test.
T.7	Satisfied. See video of test.
T.8	Satisfied. See video of test.
T.9	Satisfied. See video of test.

Appendix A. Logic Flow Chart

Figure 2. Logic flow for program.



APPENDIX B. DESIGN CODE

C/C++

```
#include <Arduino.h>
#include <limits.h>


/* Custom register definitions for the ATmega328P */
#define MY_TCCR2A (*(volatile unsigned char *)0xB0)
#define MY_TCCR2B (*(volatile unsigned char *)0xB1)
#define MY_TCNT2  (*(volatile unsigned char *)0xB2)
#define MY_OCR2A  (*(volatile unsigned char *)0xB3)
#define MY_TIMSK2 (*(volatile unsigned char *)0x70)
#define MY_TIFR2  (*(volatile unsigned char *)0x37)
#define MY_SREG   (*(volatile unsigned char *)0x5F)

/* Custom Timer1 registers for fan PWM on pin 9 */
#define MY_TCCR1A (*(volatile unsigned char *)0x80)
#define MY_TCCR1B (*(volatile unsigned char *)0x81)
#define MY_TCNT1L (*(volatile unsigned char *)0x84)
#define MY_TCNT1H (*(volatile unsigned char *)0x85)
#define MY_OCR1AL (*(volatile unsigned char *)0x88)
#define MY_OCR1AH (*(volatile unsigned char *)0x89)
#define MY_TIMSK1 (*(volatile unsigned char *)0x6F)
#define MY_PORTB  (*(volatile unsigned char *)0x25)


volatile unsigned long myMilliCounter = 0;


int selectedLED = -1;


unsigned long blinkRateOne = 0;
unsigned long blinkRateTwo = 0;


unsigned long blinkTrackerOne = 0;
unsigned long blinkTrackerTwo = 0;


unsigned long inputValue = 0;
int inputStarted = 0;
int inputInvalid = 0;


/* Fan PWM output and button input */
const int fanPin = 9;
const int buttonPin = 4;

/* OFF, LOW, MEDIUM, HIGH, MEDIUM, LOW; then repeat */
const int fanSpeeds[6] = {0, 85, 170, 255, 170, 85};
int fanStep = 0;

/* Button debouncing */
int lastButtonReading = HIGH;
int stableButtonState = HIGH;
```

```
unsigned long buttonTracker = 0;
const unsigned long debounceTime = 30;


/* Custom Timer2 compare-match interrupt */
extern "C" void __vector_7(void)
    __attribute__((signal, used, externally_visible));


void __vector_7(void)
{
  myMilliCounter++;
}


/* Returns a copy of the millisecond */
unsigned long myMilli(void)
{
  unsigned char savedStatus;
  unsigned long currentTime;


  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");


  currentTime = myMilliCounter;


  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");


  return currentTime;
}


/* Configures Timer2 to interrupt once every millisecond */
void setupMyMilli(void)
{
  unsigned char savedStatus;


  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");


  MY_TCCR2B = 0;       /* Stop Timer2 */
  MY_TIMSK2 = 0;       /* Disable Timer2 interrupts */
  MY_TCCR2A = 0x02;    /* Clear timer on compare match */
  MY_TCNT2 = 0;
  MY_OCR2A = 249;      /* 250 counts per interrupt */
  MY_TIFR2 = 0x07;     /* Clear old interrupt flags */


  myMilliCounter = 0;
```

```
  MY_TIMSK2 = 0x02;    /* Enable compare-match A interrupt */
  MY_TCCR2B = 0x04;    /* Start Timer2 with prescaler 64 */


  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");
}


/* Processes a complete number after Enter is pressed */
void processInput(void)
{
  unsigned long currentTime;


  if (inputInvalid) {
    Serial.println("Invalid input. Enter a whole number.");
  }
  else if (selectedLED == -1) {
    if (inputValue == 1 || inputValue == 2) {
      selectedLED = (int)inputValue;
    }
    else {
      Serial.println("Please choose LED 1 or 2.");
    }
  }
  else {
    currentTime = myMilli();


    if (selectedLED == 1) {
      blinkRateOne = inputValue;
      blinkTrackerOne = currentTime;
      digitalWrite(2, LOW);
    }
    else {
      blinkRateTwo = inputValue;
      blinkTrackerTwo = currentTime;
      digitalWrite(3, LOW);
    }


    selectedLED = -1;
  }


  inputValue = 0;
  inputStarted = 0;
  inputInvalid = 0;


  if (selectedLED == -1) {
    Serial.println("What LED? (1 or 2):");
  }
  else {
    Serial.println("What interval (in msec):");
```

```
  }
}


/* Reads one character at a time so LEDs keep blinking */
void taskSerial(void)
{
  char character;
  unsigned long digit;


  if (Serial.available() <= 0) {
    return;
  }


  character = (char)Serial.read();


  if (character == '\r' || character == '\n') {
    if (inputStarted) {
      processInput();
    }
  }
  else if (character >= '0' && character <= '9') {
    inputStarted = 1;
    digit = (unsigned long)(character - '0');


    /* Prevent the input number from overflowing */
    if (inputValue > (ULONG_MAX - digit) / 10UL) {
      inputInvalid = 1;
    }
    else {
      inputValue = inputValue * 10UL + digit;
    }
  }
  else {
    inputStarted = 1;
    inputInvalid = 1;
  }
}


/* Sets fan output: 0 = OFF, 255 = fully ON */
void setFanSpeed(int speed)
{
  unsigned char savedStatus;

  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");

  /* Write the high byte before the low byte */
  MY_OCR1AH = 0;
  MY_OCR1AL = (unsigned char)speed;

  if (speed == 0) {
```

```
    /* Disconnect PWM and hold pin 9 LOW */
    MY_TCCR1A = 0x01;
    MY_PORTB &= 0xFD;
  }
  else if (speed == 255) {
    /* Disconnect PWM and hold pin 9 HIGH */
    MY_TCCR1A = 0x01;
    MY_PORTB |= 0x02;
  }
  else {
    /* Connect non-inverting PWM to pin 9 */
    MY_TCCR1A = 0x81;
  }

  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");
}


/* Configures Timer1 for 8-bit phase-correct PWM */
void setupFan(void)
{
  unsigned char savedStatus;

  savedStatus = MY_SREG;
  __asm__ __volatile__("cli" ::: "memory");

  MY_TCCR1B = 0;       /* Stop Timer1 during configuration */
  MY_TIMSK1 = 0;       /* Hardware PWM needs no Timer1 ISR */
  MY_TCCR1A = 0x01;    /* 8-bit phase-correct mode; output disconnected */

  /* Clear the counter and duty-cycle registers */
  MY_TCNT1H = 0;
  MY_TCNT1L = 0;
  MY_OCR1AH = 0;
  MY_OCR1AL = 0;

  MY_PORTB &= 0xFD;    /* Start with pin 9 LOW: fan OFF */
  MY_TCCR1B = 0x03;    /* Start Timer1 with prescaler 64 */

  MY_SREG = savedStatus;
  __asm__ __volatile__("" ::: "memory");
}


/* Changes fan speed once per debounced button press */
void taskFan(void)
{
  int buttonReading;
  unsigned long currentTime;

  buttonReading = digitalRead(buttonPin);
  currentTime = myMilli();

  /* Restart debounce timing whenever the input changes */
  if (buttonReading != lastButtonReading) {
    buttonTracker = currentTime;
```

```
  }

  /* Accept the new state after it stays unchanged for 30 ms */
  if (currentTime - buttonTracker >= debounceTime) {
    if (buttonReading != stableButtonState) {
      stableButtonState = buttonReading;

      /* INPUT_PULLUP makes a pressed button read LOW */
      if (stableButtonState == LOW) {
        fanStep++;

        if (fanStep >= 6) {
          fanStep = 0;
        }

        setFanSpeed(fanSpeeds[fanStep]);
      }
    }
  }

  lastButtonReading = buttonReading;
}


void setup(void)
{
  Serial.begin(9600);


  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);

  pinMode(fanPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);


  digitalWrite(2, LOW);
  digitalWrite(3, LOW);

  /* Configure custom PWM and start the fan OFF */
  setupFan();


  setupMyMilli();


  Serial.println("What LED? (1 or 2):");
}


void loop(void)
{
  unsigned long currentTime;
  unsigned long halfInterval;


  currentTime = myMilli();
```

```
  /* LED 1 */
  if (blinkRateOne > 0) {
    halfInterval = blinkRateOne / 2;


    if (halfInterval == 0) {
      halfInterval = 1;
    }


    if (currentTime - blinkTrackerOne >= halfInterval) {
      digitalWrite(2, !digitalRead(2));
      blinkTrackerOne = currentTime;
    }
  }


  /* LED 2 */
  if (blinkRateTwo > 0) {
    halfInterval = blinkRateTwo / 2;


    if (halfInterval == 0) {
      halfInterval = 1;
    }


    if (currentTime - blinkTrackerTwo >= halfInterval) {
      digitalWrite(3, !digitalRead(3));
      blinkTrackerTwo = currentTime;
    }
  }


  taskSerial();
  taskFan();
}
```

END OF DOCUMENT